

# LifeLink - Blood Donation Management System

## Abstract:

This project delivers a robust **Blood Donation Management System** named **LifeLink**, developed in C++ using a console-based interface. It effectively manages donor records, recipient requests, and the blood inventory in a non-persistent environment (simulating an operational bank). The system extensively applies core **Object-Oriented Programming (OOP) principles**, including **inheritance** (Person --> Donor, Person --> Recipient), **polymorphism** (virtual functions and destructors), **operator overloading** (for donor tracking, inventory math, and output streams), **templates** (for the generic BloodBank container), and **friend functions**. The application features basic file handling for data persistence and demonstrates a clean, functional design suitable for managing a blood donation database.

## Objective:

- To demonstrate comprehensive understanding and practical implementation of **OOP concepts**, including classes, inheritance, polymorphism, templates, operator overloading, and file handling.
- To create a functional system for **managing donor and recipient records** with unique IDs and tracking essential data (age, blood group, contact).
- To implement a **Blood Inventory** system that accurately tracks stock levels for all major blood groups and handles transactions (donations and requests).
- To ensure data persistence for donors, recipients, and inventory across sessions using simple **text file I/O**.
- To utilize **STL containers** (vector, map) and **utility functions** to efficiently store, retrieve, and process records.

## Introduction:

Blood donation management system is a critical process requiring accurate record-keeping and efficient inventory tracking. The LifeLink system provides a simplified, digital solution for managing these operations. Implemented entirely in **C++**, this project serves as a hands-on demonstration of structuring a real-world application using modular, object-oriented principles. It offers functionality to register new donors and recipients, record donations, fulfill requests, and maintain a clear, real-time view of the blood inventory.

## Technology Stack:

- **Language:** C++
- **Development Environment:** Compatible with standard C++ compilers (GCC/G++).
- **File Handling:** Text file operations (fstream).
- **Libraries Used:** Standard Template Library (**STL**), iostream, fstream, iomanip, ctime.

## Key Features:

- **Inheritance Hierarchy:** Person base class for common attributes (name, age, blood group, contact).
- **Polymorphism:** Virtual display() function and **virtual destructor** in Person.
- **Template Class:** BloodBank<T> provides a generic container for managing lists of both Donor and Recipient records.
- **Static Members:** Donor :: donorCount tracks the total number of registered donors.
- **Overloaded Operators:** Used for record display (operator<<), tracking donations (operator++), and inventory management (operator+, operator-)
- **Data Persistence:** Records and inventory stock are saved to and loaded from text files (donors.txt, recipients.txt, inventory.txt).
- **Blood Inventory Management:** Tracks 8 major blood groups (A+, A-, B+, B-, AB+, AB-, O+, O-).

## User Interface

### LifeLink Main menu

```
WELCOME TO LIFELINK
Blood Donation Management System

LIFELINK - BLOOD DONATION SYSTEM

1. Register Donor
2. Record Donation
3. Register Recipient
4. Fulfill Blood Request
5. View All Donors
6. View All Recipients
7. Search Donors by Blood Group
8. View Blood Inventory
9. Exit

Enter choice: █
```

### Donor registration

```
Enter choice: 1

===== REGISTER NEW DONOR =====
Name: Mahmud
Age: 23
Blood Group (A+/A-/B+/B-/AB+/AB-/O+/O-): B+
Contact: 01757799786

✓ Donor registered successfully! ID: 1007
```

### Record Donation Success

```
Enter choice: 2
Enter Donor ID: 1007

✓ Donation recorded successfully!
ID: 1007 | Name: Mahmud | Age: 23 | Blood: B+ | Contact: 01757799786 | Last Donation: 24/11/2025 | Total: 1
```

### View Blood Inventory

```
Enter choice: 8

===== BLOOD INVENTORY =====
Blood Group Units Available
-----
A+          3
A-          0
AB+         1
AB-         0
B+          4
B-          3
O+          1
O-          0
=====
```

## View All Donors

```
Enter choice: 5

===== ALL DONORS =====
ID: 1001 | Name: Rafi | Age: 22 | Blood: B+ | Contact: 45673 | Last Donation: 24/11/2025 | Total: 5
ID: 1002 | Name: Hasan | Age: 22 | Blood: A+ | Contact: 45768 | Last Donation: 24/11/2025 | Total: 4
ID: 1003 | Name: Murad | Age: 34 | Blood: AB+ | Contact: 56372 | Last Donation: 24/11/2025 | Total: 2
ID: 1004 | Name: Kamal | Age: 42 | Blood: B- | Contact: 38293 | Last Donation: 24/11/2025 | Total: 4
ID: 1005 | Name: Prome | Age: 22 | Blood: O+ | Contact: 01482 | Last Donation: 24/11/2025 | Total: 2
ID: 1006 | Name: Wasif | Age: 22 | Blood: O+ | Contact: 29193 | Last Donation: Never | Total: 0
ID: 1007 | Name: Mahmud | Age: 23 | Blood: B+ | Contact: 01757799786 | Last Donation: 24/11/2025 | Total: 1

Total Donors: 7
```

## View All Recipients

```
Enter choice: 6

===== ALL RECIPIENTS =====
ID: 2001 | Name: Jalil | Age: 56 | Blood: O+ | Contact: 10382 | Request Date: | Units: 1 | Status: Fulfilled
ID: 2002 | Name: Shohagh | Age: 48 | Blood: A+ | Contact: 10382 | Request Date: | Units: 2 | Status: Pending
```

## Fulfill Request Failure

```
Enter choice: 4

Enter Recipient ID: 2003

x Insufficient blood units! Available: 1
```

## Recipient Registration

Enter choice: 3

===== REGISTER RECIPIENT =====

Name: Galib

Age: 45

Blood Group: AB+

Contact: 01484

Units Needed: 3

✓ Recipient registered! ID: 2003

## Search Donors

LIFELINK - BLOOD DONATION SYSTEM

1. Register Donor
2. Record Donation
3. Register Recipient
4. Fulfill Blood Request
5. View All Donors
6. View All Recipients
7. Search Donors by Blood Group
8. View Blood Inventory
9. Exit

Enter choice: 7

Enter Blood Group: B+

===== DONORS WITH B+ =====

ID: 1001 | Name: Rafi | Age: 22 | Blood: B+ | Contact: 45673 | Last Donation: 24/11/2025 | Total: 5

ID: 1007 | Name: Mahmud | Age: 23 | Blood: B+ | Contact: 01757799786 | Last Donation: 24/11/2025 | Total: 1

## Data Persistence

9. Exit

Enter choice: 9

Thank you for using LifeLink!

Saving data before exit...

Donors saved successfully!

Recipients saved successfully!

## Class Inheritance and Relationships:

The system structure is based on a clear object hierarchy and composition:

### Class Hierarchy:

| Relationship | Base Class   | Derived Class                           | Purpose                                                      |
|--------------|--------------|-----------------------------------------|--------------------------------------------------------------|
| Inheritance  | Person       | Donor                                   | Adds ID, donation history, and static count.                 |
| Inheritance  | Person       | Recipient                               | Adds ID, request date, units needed, and fulfillment status. |
| Template     | BloodBank<T> | BloodBank<Donor> / BloodBank<Recipient> | Generic container for managing records.                      |

## Template Hierarchy Structure

### 1. Template Class

- **BloodBank<typename T>**: This class is the core of the template architecture. It is designed to manage collections of any type T that inherits from Person.
  - **Data Member:** vector<T\*> records; (Stores pointers to objects of type T).
  - **Methods:** addRecord(T\*), getAllRecords(), findByBloodGroup(string), getCount().

### 2. Template Instantiations

LifeLink creates two **concrete** instances of the BloodBank template, allowing it to manage two distinct types of records using the exact same underlying logic:

| Concrete Class       | Template Type (T) | Purpose                                          |
|----------------------|-------------------|--------------------------------------------------|
| BloodBank<Donor>     | Donor             | Stores and manages all <b>Donor</b> records.     |
| BloodBank<Recipient> | Recipient         | Stores and manages all <b>Recipient</b> records. |

### 3. Template Utility Functions

LifeLink also uses **template functions** to perform actions on these generic collections, enhancing code reusability:

| Template Function                                       | Purpose                                                                                                                                                              |
|---------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>displayAll&lt;T&gt;(const vector&lt;T*&gt;&amp;)</b> | Iterates over any collection (Donor or Recipient) and uses the type-specific overloaded operator<< for clean output.                                                 |
| <b>findById&lt;T&gt;(vector&lt;T*&gt;&amp;, int)</b>    | Searches a collection for a record by its unique ID. It uses C++17's <b>if constexpr</b> to safely call either getDonorID() or getRecipientID() based on the type T. |

This design allows to add new classes (e.g., Staff, Volunteer) that inherit from Person and immediately use a new BloodBank<Staff> container without writing any new management code.

#### ***Friend Relationships:***

- **Friend Function:** updateDonation is a friend of Donor to efficiently update lastDonationDate and increment totalDonations (++donor) simultaneously.

#### ***Association/Composition:***

- **LifeLink has a BloodBank<Donor>** (Composition)
- **LifeLink has a BloodBank<Recipient>** (Composition)
- **LifeLink has a BloodInventory** (Composition)

#### **Code Overview:**

| Class Implemented | Key Functionality                                          | OOP Concept Demonstrated                                              |
|-------------------|------------------------------------------------------------|-----------------------------------------------------------------------|
| <b>Person</b>     | Base class for shared details.                             | Inheritance, Virtual Destructor/Function                              |
| <b>Donor</b>      | Tracks donation history, total donations, and ID.          | Inheritance, Operator Overloading (++ , == , int , <<), Static Member |
| <b>Recipient</b>  | Tracks units needed, request date, and fulfillment status. | Inheritance, Operator Overloading (<<)                                |

|                           |                                                  |                                         |
|---------------------------|--------------------------------------------------|-----------------------------------------|
| <b>BloodBank&lt;T&gt;</b> | Generic container for storing records of type T. | <b>Template Class</b> , STL (vector)    |
| <b>BloodInventory</b>     | Manages stock levels for all 8 blood groups.     | STL (map), Operator Overloading (+, -)  |
| <b>FileHandler</b>        | Static methods for saving/loading data.          | File Handling, Static Methods           |
| <b>LifeLink</b>           | Main application logic and user interface.       | Aggregation/Composition of core classes |

## Core OOP Implementations:

### Overloaded Operators:

| Operator                         | Class            | Usage                                                         |
|----------------------------------|------------------|---------------------------------------------------------------|
| operator<< (Stream Insertion)    | Donor, Recipient | Enables easy, formatted printing of objects to cout.          |
| operator++ (Prefix Increment)    | Donor            | Increments totalDonations when a donor makes a new donation.  |
| operator+, operator- (Binary)    | BloodInventory   | Allows mathematical operations between two inventory objects. |
| operator== (Equality)            | Donor            | Compares donors based on donorID.                             |
| operator int() (Type Conversion) | Donor            | Converts a Donor object to its totalDonations count.          |

### Virtual Functions:

| Virtual Function             | Location      | Purpose                                                                                                                                 |
|------------------------------|---------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| virtual ~Person()            | Person (Base) | <b>Virtual Destructor:</b> Ensures Donor and Recipient memory is correctly freed when deleted via a Person* pointer from the BloodBank. |
| virtual void display() const | Person (Base) | Provides a common interface for displaying derived objects (Overridden in Donor and Recipient).                                         |



### STL Containers Used:

- **vector<T\*>**: Used inside BloodBank<T> to store dynamic arrays of Donor or Recipient pointers.
- **map<string, int>**: Used in BloodInventory to map blood group strings (e.g., "A+") to their integer unit counts for fast lookup and modification.

### Discussion:

The LifeLink project successfully applies a wide array of Object-Oriented Programming concepts to solve a practical data management problem. The use of the **Template Class** BloodBank<T> effectively eliminates duplicate code for managing two similar, but distinct, sets of records (Donor and Recipient). **Polymorphism** is correctly implemented using the virtual destructor, which is essential for safely managing the memory allocated in the BloodBank container.

A key design choice was using a **Friend Function** (updateDonation) to couple two related operations (updating the donation date and incrementing the total count) while maintaining encapsulation for the Donor's internal data. The system relies on simple text file I/O for data persistence, which is straightforward but limited in terms of security and performance.

### Future Enhancements:

- Implement a **BloodManager** class to handle complex blood type compatibility and matching requests.
- Upgrade file handling to use **binary files** for faster loading/saving and better security.
- Introduce **Error Handling** using try-catch blocks for file operations and invalid user inputs (e.g., non-integer age).
- Add a **reporting feature** to generate structured summaries of low stock or frequent donors.
- Integrate a unique **Donor ID generation** logic that reads the last ID from the persistent storage file.

## References

1. **E. Balagurusamy** - *Object-Oriented Programming with C++*
2. **Course Materials** - *Object-Oriented Programming Class Lectures and Lab Slides.*
3. **C++ Documentation** - Used for Standard Template Library (STL), Input/Output (fstream), and Time functions (ctime).
4. **GeeksforGeeks** - *Consulted for C++ code snippets, conceptual clarity on OOP topics, and algorithm implementation.*
5. **C++ Programming Guides** (Online Resources) - Referenced for best practices in Object-Oriented Design (OOD).